



Slides against P2971R3

No implication for C++

Document number:

[P3736R0](#)

Date:

2025-06-27

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Reply-To:

Jan Schultke <janschultke@gmail.com>

Source:

github.com/Eisenwave/cpp-proposals/blob/master/src/implication-slides.cow



IMPORTANT: This document has custom controls:

-   : go to the next slide
-   : go to previous slide

No implication for C++

P2971R3

Comparison

Status quo	P29271R3
<pre>// from P2971R3: reference_type<D> destructible_type<D>; not x.has_value() *x == *y; // Mine: p == nullptr p->valid;</pre>	<pre>// from P2971R3: nonreference_type<D> => destructible_type<D>; x.has_value() => *x == *y; // Mine: p != nullptr => p->valid;</pre>

- P29271R3 hinges on right side being *much* better, enough for new operator
- that is obviously not the case

Adding operators is costly

- more ways to express same code
- operator precedence rules to learn
- teaching materials, compilers, tooling, etc.
- valuable syntax space is lost forever
 - *especially* '`=>`' could be used in the future (abbreviated lambdas?)
- smaller common subset with other languages
- unlike `||`, `&&`, and `!`, no `or`, `and`, and `not`
 - not realistic to add `implies`/`impl` alt. keyword

Confusing fold expressions

Given `(args => ...)`:

- for one-element pack, equivalent to `true => args`
 - suggests that neutral element is `true`
- for zero-element pack, equivalent to `false`
 - suggested by author for symmetry with `or`



IMPORTANT: This is not a problem worth solving. `and` and `or` do this much better already!

Implication invites bugs

```
// i is equal or greater to 0... or is it?  
for (int i = 10; i => 0; --i) {  
    // ...  
}
```

- currently: compiler error
- after P2971R3: does the wrong thing